

Discrete Diffusion-Based Model-Level Explanation of Heterogeneous GNNs with Node Features

Pallabee Das
Paderborn University
Paderborn, Germany
pdas@mail.uni-paderborn.de

Stefan Heindorf
Paderborn University
Paderborn, Germany
heindorf@uni-paderborn.de

Abstract

Many real-world datasets, such as citation networks, social networks, and molecular structures, are naturally represented as heterogeneous graphs, where nodes belong to different types and have additional features. For example, in a citation network, nodes representing “Paper” or “Author” may include attributes like keywords or affiliations. A critical machine learning task on these graphs is node classification, which is useful for applications such as fake news detection, corporate risk assessment, and molecular property prediction. Although Heterogeneous Graph Neural Networks (HGNNs) perform well in these contexts, their predictions remain opaque. Existing post-hoc explanation methods lack support for actual node features beyond one-hot encoding of node type and often fail to generate realistic, faithful explanations. To address these gaps, we propose DiGNNExplainer, a model-level explanation approach that synthesizes heterogeneous graphs with realistic node features via discrete denoising diffusion. In particular, we generate realistic discrete features (e.g., bag-of-words features) using diffusion models within a discrete space, whereas previous approaches are limited to continuous spaces. We evaluate our approach on multiple datasets and show that DiGNNExplainer produces explanations that are realistic and faithful to the model’s decision-making, outperforming state-of-the-art methods.

CCS Concepts

• **Computing methodologies** → *Supervised learning by classification; Neural networks.*

Keywords

Graph neural networks; Explainable artificial intelligence; Diffusion models

ACM Reference Format:

Pallabee Das and Stefan Heindorf. 2026. Discrete Diffusion-Based Model-Level Explanation of Heterogeneous GNNs with Node Features. In *Proceedings of the ACM Web Conference 2026 (WWW ’26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792518>

1 Introduction

Complex real-world data, such as citation networks, social networks, and molecular structures, often exhibit an underlying graph

structure. Nodes in these graphs typically have different types and are associated with additional features, i.e., they are heterogeneous graphs. For instance, in a citation network, there might be nodes of type “Paper” and “Author” each of which has additional node features like keywords or affiliations. Whereas related work sometimes uses the term “node feature” only to refer to a one-hot encoding of the node type [27, 31], we refer to actual attributes of nodes (e.g., age of persons, keywords of papers). A key machine learning task on such graphs is node classification, e.g., for fake news detection [33], corporate risk assessment [34], or predicting the mutagenicity of molecules [18]. Towards this end, a number of powerful approaches for heterogeneous graphs have been proposed, e.g., Heterogeneous Graph Neural Networks (HGNNs) [26].

However, the predictions made by HGNNs are opaque, providing no insight into the reasoning behind their decisions. In high-stakes applications—such as healthcare, finance, or criminal justice—this lack of transparency is a significant limitation, as it prevents users from understanding the rationale behind the model’s decisions. Towards this end, several post-hoc explanation methods have been proposed, such as GNNExplainer [29], SubgraphX [32], PGExplainer [18], XGNN [31], GNNInterpreter [27], and D4Explainer [4]. Despite their contributions, these methods face significant limitations: (1) the first three do not provide model-level explanations, (2) the second three lack support for heterogeneous graphs and node features, (3) they are not designed to generate plausible, realistic graphs as explanations. Although node feature support has been identified as important future work [4], most explainers still lack it. This is a critical gap, as GNNs tend to perform poorly without node features [7].

In this paper, we introduce DiGNNExplainer, a model-level explanation approach for heterogeneous GNNs that synthesizes realistic heterogeneous graphs with *node features* (Figure 1). Our approach consists of five key steps: (1) We generate realistic graph structures using discrete denoising diffusion with DiGress [25]. (2) We generate realistic node features using discrete denoising diffusion. Specifically, we extend TabDDPM [16], a diffusion model originally designed for tabular data, to synthesize realistic, discrete node features—yielding a novel feature generator dubbed DiTabDDPM. Both our graph generator and our feature generator are trained on the same dataset as the GNN, keeping the generated graphs in-distribution, i.e., they reflect the patterns and statistical properties of the training data. (3) We check whether the synthesized graphs adhere to the metagraph and discard graphs that do not. (4) For each class, we select the synthetic graph with node features that maximizes the GNN’s prediction score for that class, ensuring that the selected graph captures the essential information relevant to the model’s decision-making. (5) We evaluate our synthesized



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW ’26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792518>

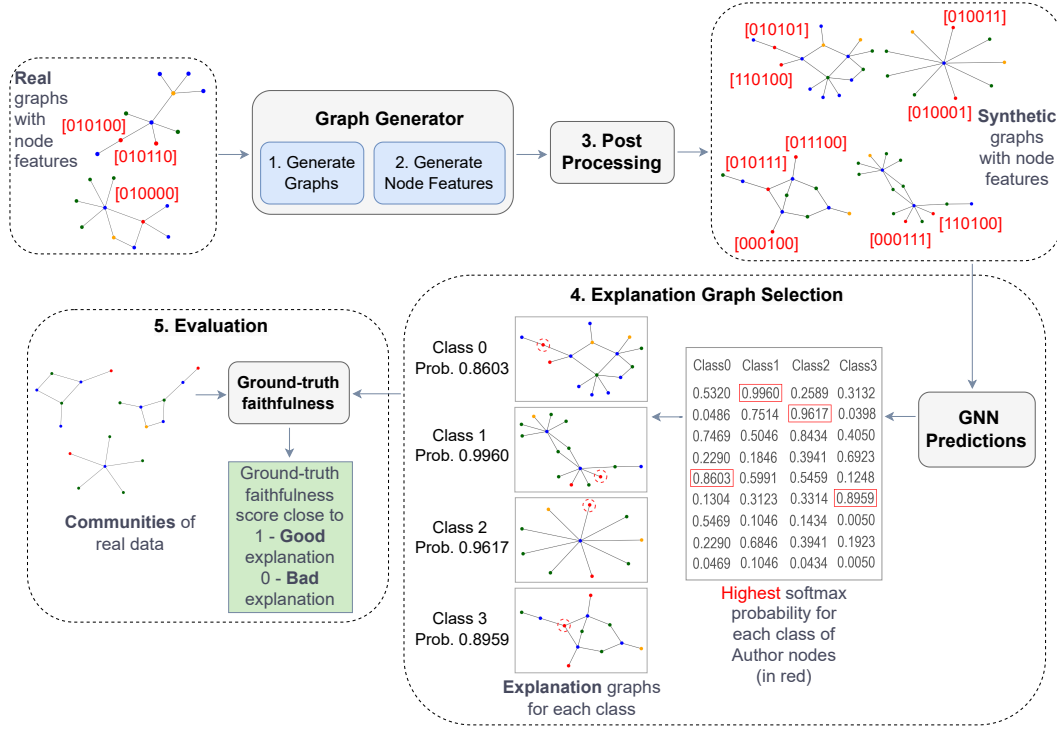


Figure 1: Overview of DiGNNEExplainer. The example, based on the DBLP dataset, illustrates the process of generating explanations for a node classification task. Nodes represent different entity types: authors (red, ●), papers (blue, ●), conferences (orange, ●), and terms (green, ●). Node features, such as discrete bag-of-words representations of author keywords (e.g., [010100]), are encoded in red. The task is to predict the class of each author, i.e., their research area: database, data mining, artificial intelligence, or information retrieval. (1) & (2) The graph generator takes as input small real-world graphs with discrete node features and produces synthetic graphs with discrete node features. (3) The validity of these generated graphs is checked against the metagraph. (4) A trained GNN, which is to be explained, is applied to each of the nine author nodes of valid, synthesized graphs, producing a prediction for each of the four classes. For each class, we identify the node and corresponding graph with the highest prediction probability, yielding one explanation graph per class. (5) To evaluate our approach, we compute various metrics, including the ground-truth faithfulness between our explanation graphs and the communities within the real dataset.

explanation graphs with node features both in terms of realism and faithfulness. Specifically, we assess whether they preserve graph distributions, e.g., node degrees, clustering coefficients, and spectrum in terms of maximum mean discrepancy (MMD), and the generated explanations are faithful, i.e., achieve a high GNN prediction score (predictive faithfulness), and retain graph communities of the original graph (ground-truth faithfulness). We compare DiGNNEExplainer to previous model-level explanation approaches and conduct an ablation study on different node feature generation techniques.

We make the following contributions:

- DiGNNEExplainer generates *model-level* explanations including *actual node features*, unlike previous methods that either lack model-level explanations or ignore node features.
- DiGNNEExplainer introduces a novel explanation pipeline as shown in Figure 1.
- We introduce a novel explanation graph generation technique, combining forest fire sampling with multiple DiGress models for different graph sizes.

- Unlike existing tabular data generators [14, 16], which map all node features into a continuous space, our novel DiTabDPM directly models discrete features using discrete diffusion, improving performance.
- Our evaluation shows that DiGNNEExplainer synthesizes realistic explanation graphs closely matching the graph statistics (Table 1) and node feature statistics of the original training dataset (Table 2), and achieves higher faithfulness—both predictive faithfulness and ground-truth faithfulness (Table 3).

2 Related Work

We briefly survey explanation methods for graph neural networks (GNNs) and generative models, which we employ to generate explanations.

Model-level GNN explainers explain why a GNN predicts a certain class by generating a graph that maximizes the predicted score for this class. Notable methods include XGNN [31], GNNInterpreter [27], and D4Explainer [4]. We compare them in terms of their graph generation, node feature generation, and evaluation.

Graph generation: XGNN employs reinforcement learning to iteratively add edges or nodes with one-hot encoded types. GNNInterpreter uses a probabilistic generative model generating edges and nodes independently with a fixed probability. D4Explainer, like our approach, utilizes diffusion-based graph generation but lacks support for different node types. **Node feature generation:** XGNN only supports one-hot encoded node types. GNNInterpreter proposes the use of actual node features but evaluates only node types. D4Explainer briefly discusses node features but focuses on structural graph generation. **Evaluation:** All three methods evaluate graphs based on predicted class scores. Only D4Explainer considers simple graph statistics; XGNN and GNNInterpreter do not. None of the approaches quantitatively assesses the graphs in terms of generated communities and distribution of generated node features. While some methods provide limited support for node features, none of them effectively address node feature generation as shown in our evaluation.

Alternative explanation strategies use description logics or natural language instead of graph structures as explanation [12, 13, 15, 20, 22], e.g., via OntoLearn [5] and EvoLearner [11]. While some of these approaches support node features, they do not generate graphs as final explanations.

Instance-level GNN explainers such as GNNExplainer [29] and SubgraphX [32] explain each GNN prediction by identifying a different subgraph, making it challenging to understand the overall GNN behavior. As pointed out by Chen et al. [4], these methods derive their subgraphs from the training datasets, leading to unreliable explanations for out-of-distribution data.

Graph generators learn the distribution of a set of graphs to generate similar graphs. GraphVAE [23] employs a variational autoencoder to embed graphs into a continuous latent space and generates new graphs by sampling from this space. DiGress [25] uses a discrete diffusion-based approach reconstructing graphs step-by-step in a discrete space, preserving structural information and node-type distributions more effectively. Both GraphVAE and DiGress have demonstrated strong performance on molecular datasets like MUTAG and QM9. HGEN [17] employs a Generative Adversarial Network (GAN) to preserve metapath relations of heterogeneous datasets, such as DBLP, PubMed, and IMDB. HGEN generates graphs with the same number of nodes as the input graph with a minimum number of 100 nodes, limiting its ability to produce small, human-intelligible graphs that may serve as an explanation. These approaches focus on structural graph generation with limited support for actual node features: HGEN does not generate node features at all, while GraphVAE and DiGress only generate discrete, categorical node features focusing on one-hot encoding of node types. Their performance for node feature generation has not been evaluated. Moreover, none of these approaches has been used and evaluated for generating explanation graphs as done in this work.

Tabular feature generation TabDDPM [16] uses diffusion models to synthesize discrete and continuous tabular data by learning a continuous space. To the best of our knowledge, it has never been used and evaluated to generate node features of graphs. Moreover, it has never been used for generating explanations. We extend TabDDPM to synthesize realistic discrete features by modeling them directly within a discrete space.

3 DiGNNExplainer

Our DiGNNExplainer provides model-level explanations for heterogeneous GNNs. To achieve this, we generate explanation graphs by sampling from the training data distribution using a graph generator (Figure 1). We use the trained GNN that we want to explain to make predictions on the synthetically generated graphs. The graph with the highest predicted probability for a class serves as the explanation for that class. Our graph generator is designed to produce plausible explanation graphs that do not need to be subgraphs of the training data.

3.1 Heterogeneous Graph with Node Features

Our explanation graph can be formally defined as a heterogeneous graph $G = (V, E, T, X)$ where V denotes the set of nodes, $E \subseteq \{\{u, v\} | u, v \in V\}$ the set of undirected edges, T the set of node types, and $X = \{X_1, \dots, X_{|T|}\}$ the set of feature matrices. Each node $v \in V$ is associated with a node type $\tau \in T$. The feature matrix for node type τ is denoted by $X_\tau \in \mathbb{R}^{N_\tau \times d_\tau}$, where N_τ is the number of nodes of type τ and d_τ the number of features per node. Each node type can either have continuous $X_\tau \in \mathbb{R}^{N_\tau \times d_\tau}$ or discrete $X_\tau \in \mathcal{X}_\tau^{N_\tau \times d_\tau}$ features where $\mathcal{X}_\tau = \{x_\tau^1, \dots, x_\tau^{k_\tau}\}$ denotes the set of allowed discrete values for type τ .

3.2 Generating Graphs and Node Features with Diffusion Models

We generate explanation graphs and node features through diffusion approaches (see Figure 1). We do so (1) using diffusion models for the graph structure and (2) diffusion models for the node features, (3) discard graphs that are inconsistent with the metagraph, and (4) we select one explanation graph per class. In the following, we give an overview of our approach. Detailed descriptions of the diffusion processes using DiGress and TabDDPM, and formal definitions of DiGNNExplainer, are provided in the Appendix.

1. Graph generation. When synthesizing graphs, our goal is twofold: first, to synthesize graphs that are plausible and match the distribution of our training data; second, to produce small, human-intelligible graphs that can serve as explanations.

We build upon DiGress [25] to synthesize heterogeneous graph structures. Since DiGress performs well in generating graphs of a fixed size n and requires training data of the same size, while the best size of an explanation graph is not known in advance, we create multiple training sets for DiGress with different graph sizes and train a different DiGress model for each graph size.¹ For graph classification, we train the graph generators directly on datasets containing multiple small graphs of different sizes. In contrast, for node classification, where the data consists of a single large graph, we extract subgraphs of varying sizes using graph sampling techniques: We employ the forest fire sampling algorithm [21], known for preserving graph connectivity and performing well in prior studies. To ensure that the sampled subgraphs include the nodes to be classified, we initiate the sampling process at nodes of the relevant types. For example, we start a fire at author nodes in the

¹In our evaluation, we typically use graph sizes between 10 and 15, as they provide meaningful explanations while remaining interpretable.

DBLP dataset. Specifically, for each graph size n , we generate k subgraphs by repeating the following process until k valid subgraphs are obtained: First, we randomly select a node of the type that is to be classified, then we start the fire from this node and stop once exactly n nodes have been burned. If fewer than n nodes are burned, we repeat the process. The nodes burned serve as sampled graphs. Following prior work, we use the default hyperparameters of the little-ball-of-fur library [21], including a burning probability of 0.4.²

2. Node feature generation. Given a node type τ with node feature matrix $\mathbf{X}_\tau$ (cf., Section 3.1), we train multiple node feature generators as follows: If τ is the node type to be classified, we train one feature generator per class to simulate different feature distributions; if τ is not the node type to be classified, we train one feature generator; if τ is a node type with no node features at all, no feature generator is trained. For example, for the DBLP dataset, we train six node feature generators: four node feature generators for the four different classes of author nodes (database, data mining, artificial intelligence, information retrieval), one for paper nodes, one for term nodes, and zero for conference nodes as they do not have node features.

If τ is a node type with a *continuous* feature matrix, we use the original TabDDPM [16] approach with continuous noise; for each node type τ with *discrete* feature matrix $\mathbf{X}_\tau$, we use our novel DiTabDDPM which combines the discrete noise from DiGress [25] and the MLPBlock architecture from TabDDPM [16] as follows:

Forward diffusion process. The forward diffusion process for node features follows DiGress [25], using discrete noise, marginal transitions, and a cosine noise scheduler.

Reverse diffusion process. In the reverse process, for each time step t and node type τ , given a discrete feature vector $\mathbf{x}_t \in \mathcal{X}_\tau^{d_\tau}$, we encode it via one-hot encoding as a vector $\mathbf{x}_t^{(\text{ohc})} \in \{0, 1\}^{d_\tau \times k_\tau}$ into the denoising network which outputs the feature distribution vector $\mathbf{p}_{t-1} \in [0, 1]^{d_\tau \times k_\tau}$ for its previous step. Then we sample the discrete features $\mathbf{x}_{t-1} \in \mathcal{X}_\tau^{d_\tau}$ of the previous time step $t-1$ from the multinomial distribution defined by $\mathbf{p}_{t-1}$ and repeat the process.

Node feature loss. The node feature training loss L_{train} for a single node is the cross-entropy

$$L_{\text{train}} := \frac{1}{d_\tau} \sum_{i=1}^{d_\tau} \text{CrossEntropy}(\mathbf{p}_{\text{pred}}^{(i)}, \mathbf{x}_{\text{true}}^{(i)})$$

where $\mathbf{p}_{\text{pred}}^{(i)} \in [0, 1]^{k_\tau}$ is the vector of predicted probabilities, and $\mathbf{x}_{\text{true}}^{(i)} \in \{x_\tau^1, \dots, x_\tau^{k_\tau}\}$ is the true discrete value of feature i in the dataset.

3. Postprocessing. We check whether the generated graph is consistent with the metagraph of the dataset. Otherwise, we discard the graph.

4. Explanation graph selection. We obtain one explanation graph per class with the highest prediction score.

²https://little-ball-of-fur.readthedocs.io/en/latest/_modules/littleballoffur/exploration_sampling/forestfiresampler.html

4 Evaluation

In this section, we evaluate how well our DiGNNExplainer explains the predictions of GNNs. After briefly introducing our evaluation setup, we compare DiGNNExplainer to three state-of-the-art GNN explanation methods. Additionally, we compare our discrete diffusion-based approach with a variant that uses variational autoencoders, and we investigate the impact of varying the number of discrete features. Our code is publicly available.³ Additional experimental results, including dataset statistics, hyperparameters, and implementation details, can be found in the Appendix.

4.1 Evaluation Setup

Datasets. We conduct experiments on seven datasets for node and graph classification [8, 19, 28, 29]—three real-world datasets (DBLP, IMDB, MUTAG) and four synthetic datasets (BA-SHAPES, TREE-CYCLE, TREE-GRIDS, BA-3MOTIF) in Table 7. DBLP and IMDB have actual node features, the other datasets not.

GNNs. We use a high-performing GNN architecture for each dataset: GraphSAGE [10] for DBLP, HAN [26] for IMDB, and three-layer GCNs for MUTAG and the synthetic datasets. To prevent overfitting, we apply early stopping with a patience of 100 epochs, halting training if the validation score does not improve within this window.

Baseline explainers. We compare DiGNNExplainer to the following state-of-the-art model-level explanation methods: (1) XGNN [31], which leverages reinforcement learning to generate explanations. We extend the original implementation to support node classification tasks. (2) GNNInterpreter [27], which improves upon XGNN by being computationally efficient. We extend it for node classification by creating class-specific embeddings using average node pooling. (3) D4Explainer [4], which employs a diffusion-based approach to generate model-level explanations for both node and graph classification tasks.

Baseline graph generators with node features. We compare our discrete diffusion-based graph generator against two baselines: Our first baseline is a variational autoencoder-based model that jointly learns the graph structure and node features. Our second baseline replaces our discrete diffusion-based node feature generator DiTabDDPM with the continuous diffusion-based generator TabDDPM. Regarding the former, for MUTAG and the synthetic datasets, we use the original implementation of GraphVAE⁴ [23] using only node types as node features. For DBLP and IMDB, we extend GraphVAE to generate explanation graphs with actual node features by concatenating one-hot encoded node types to actual node features. We omit GraphVAE's support for node order invariance as it does not scale to graphs of our size. As GraphVAE requires all nodes to have the same number of features, but DBLP has a varying number of features per node type, we select a feature size of 50 (using feature selection) for all node types of DBLP, to be able to compare all our DBLP experiments with the baseline. For both the VAE and TabDDPM baselines, which always operate in a continuous feature

³<https://github.com/ds-jrg/DiGNNExplainer>

⁴<https://github.com/deepfindr/gvae>

space, we discretize features that are supposed to be discrete (e.g., bag-of-word features) via a threshold.

Evaluation metrics. We evaluate our approach in terms of three aspects: whether the generated graph structures have a similar distribution as the training set, whether the node features have a similar distribution as the training set, and whether our explanations are faithful.

Regarding the graph structure, following Chen et al. [4], we compare the distribution of node degrees, clustering coefficients, and spectrum in terms of maximum mean discrepancy (MMD). Additionally, we consider the MMD of the node type distribution as we are dealing with heterogeneous graphs.

To assess the similarity between synthetically generated node feature vectors $\mathbf{X}_{\text{gen}}$ and those from the training set $\mathbf{X}_{\text{train}}$, we compute the average pairwise cosine similarity $\frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n \cos(\mathbf{x}_{\text{gen}}^{(i)}, \mathbf{x}_{\text{train}}^{(j)})$, where $\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$, and $\mathbf{x}_{\text{gen}}^{(i)} \in \mathbf{X}_{\text{gen}}$, $\mathbf{x}_{\text{train}}^{(j)} \in \mathbf{X}_{\text{train}}$.

To assess *faithfulness*, following [1], we assess both *predictive* faithfulness PF and *ground-truth* faithfulness GF : Regarding the former, we compute the average class score across all classes as $PF := \frac{1}{C} \sum_{i=1}^C P(G_i)$ where C is the number of classes, G_i is our explanation graph for class i , and $P(G_i)$ is the GNN’s predicted probability for class i on explanation graph G_i . A higher score indicates greater alignment between the explanation graphs and the GNN’s original decision. Regarding the latter, we compute ground-truth faithfulness as follows: For each class, we assume that we have a set of ground-truth explanation motifs (which are readily available for synthetic datasets and can be obtained via community detection for real-world datasets as described in the appendix). Then we define $GF := \frac{1}{C} \sum_{i=1}^C \frac{1}{M_i} \sum_{j=1}^{M_i} \mathbb{1}_{(M_{i,j} \subseteq G_i)}$ where M_i is the number of available ground-truth motifs for class i , G_i is the explanation graph for class i , and $\mathbb{1}_{(M_{i,j} \subseteq G_i)}$ is the indicator function that equals 1 if the motif $M_{i,j}$ is a subgraph of the explanation graph, and 0 otherwise. A higher ground-truth faithfulness score indicates that our generated explanations are closer to the ground-truth explanations.

Train-test splits. For the node classification datasets DBLP, IMDB, BA-SHAPES, TREE-CYCLE, and TREE-GRIDS, we synthesize 1,536 graphs per dataset, some of which may be discarded as they are inconsistent with the metagraph. We use 200 graphs to train the GNN, splitting them into 128 for training, 32 for validation, and 40 for testing the graph generation model. For MUTAG (58 graphs), we allocate 38 for training, 9 for validation, and 11 for testing. For BA-3MOTIF (323 graphs), the split is 206 for training, 52 for validation, and 65 for testing. To generate candidate explanation graphs, we use our graph generation model to produce 256 graphs for each of the 6 graph sizes (either 10 to 15 nodes or 5 to 10 nodes), resulting in 1,536 candidate graphs in total.

4.2 Realism of Graph Structure

In this section, we identify the approach for generating the most realistic explanation graphs. We compare the graph distributions of the explanations generated by DiGNNEExplainer, VAE, and the baseline explainers (Table 1) with real graphs.

Implementation. (1) We generate graphs that serve as explanations. In our experiments, we restrict the explanation graph size to a range of 5 to 15 nodes to ensure human-intelligible explanation graphs. (2) We compare the average MMD (Table 1) between the explanation graphs of each class and the real graphs for node type distribution, node degree, clustering coefficient, and spectrum distributions for 50 graphs.

Results. DiGNNEExplainer achieves the highest similarity for node degree distribution across all datasets except BA-SHAPES, and the highest similarity for node type distribution (NTD) and clustering coefficient (Clus.) for the real-world datasets (Table 1).

Interpretation. DiGNNEExplainer generates the most realistic graphs, preserving the distributions of graphs for all datasets and node types for heterogeneous datasets. The homogeneous datasets have a single node type, hence the comparison always yields a score of 0.0.

4.3 Realism of Node Features

In this section, we identify the approach for generating the most realistic node features using cosine similarity (Node feat. (cos) in Table 2). We compare DiTabDDPM and TabDDPM in terms of node feature realism for various node feature sizes. The notations used for our feature selection experiments in the Table 2 are defined as: (i) DiTabDDPM—discrete (bag-of-words) node features generated using DiTabDDPM and continuous node features generated using TabDDPM (ii) TabDDPM—all node features generated using TabDDPM (iii) DiTabDDPM (8 discr. feat.) indicates that the node to be classified has a feature size of 8.

Implementation. (1) For discrete node features, we use one-hot encoding of word frequencies, i.e., in the range of 0 to 5 for paper nodes. (2) For VAE, we learn the graphs along with node features, which include all features, i.e., 50 for DBLP and 3066 for IMDB. In VAE, we average the cosine similarity of the node features of the node to be classified for 50 graphs.

Results. Cosine similarity (Node feat.) results are higher for DiTabDDPM compared to TabDDPM, with the highest score of 0.397 for DBLP.

Interpretation. DiTabDDPM, which learns a discrete space for discrete features, generates more realistic node features compared to the original TabDDPM, which learns a continuous space for discrete features.

4.4 Faithfulness of Explanations

The goal of this section is to determine which approach produces the most faithful explanations in terms of (1) predictive faithfulness and (2) ground-truth faithfulness.

In Table 3, we compare the faithfulness of our approach against existing model-level explanation approaches that do not utilize actual node features. For this comparison, we use the original homogeneous datasets from the original baseline explainer implementations. Across all settings, we keep the generated graphs and ground-truth motifs fixed and vary only the node feature sizes.

In Table 2, we train DiTabDDPM, TabDDPM, and VAE to generate heterogeneous explanation graphs with varying node feature sizes

Table 1: Comparisons for average MMD distances between explanation graphs of each class and the reference graphs, for node type distributions (NTD), node degree distributions (Deg.), clustering coefficients (Clus.), spectrum distributions (Spec.).

Approaches	DBLP				IMDB				MUTAG				BA-SHAPES				TREE-CYCLE				TREE-GRIDS				BA-3MOTIF			
	NTD↓	Deg.↓	Clus.↓	Spec.↓	NTD↓	Deg.↓	Clus.↓	Spec.↓	NTD↓	Deg.↓	Clus.↓	Spec.↓	NTD↓	Deg.↓	Clus.↓	Spec.↓	NTD↓	Deg.↓	Clus.↓	Spec.↓	NTD↓	Deg.↓	Clus.↓	Spec.↓	NTD↓	Deg.↓	Clus.↓	Spec.↓
XGNN	0.708	0.802	0.983	0.054	1.065	1.138	1.122	0.064	0.0	0.216	0.853	0.080	0.0	0.103	0.138	0.041	0.0	0.239	0.378	0.239	0.0	0.172	0.880	0.047	0.0	0.174	0.247	0.060
GNNINTERPRETER	0.708	0.946	1.311	0.077	1.065	0.730	1.161	0.058	0.0	0.240	1.002	0.071	0.0	1.223	1.070	0.043	0.0	1.261	1.480	0.047	0.0	0.976	1.405	0.046	0.0	1.411	1.251	0.068
D4EXPLAINER	0.708	0.493	0.001	0.111	1.065	0.356	0.806	0.051	0.0	0.603	0.033	0.587	0.0	0.980	1.037	0.042	0.0	0.644	0.829	0.046	0.0	0.625	1.181	0.043	0.0	1.371	0.968	0.066
VAE	0.121	0.136	0.0	0.570	0.357	0.205	0.0	0.590	0.0	0.683	1.031	0.062	0.0	1.319	1.006	1.021	0.0	1.484	1.660	0.778	0.0	0.860	1.404	0.296	0.0	0.768	0.652	0.641
DiGNNEXPLAINER	0.023	0.096	0.0	0.556	0.011	0.062	0.0	0.288	0.0	0.213	0.0	1.042	0.0	0.404	0.891	1.021	0.0	0.174	0.777	1.026	0.0	0.129	0.060	0.258	0.0	0.113	0.272	0.728

Table 2: Feature selection experiments for DBLP and IMDB. We report the average cosine similarity for node features (Node feat. (cos)), and the mean and variance of predictive faithfulness (PF) and ground-truth faithfulness (GF) over 10 runs. We compare our proposed DiTabDDPM with the original TabDDPM and the VAE baseline.

Approach	Node feat. (cos)↑	PF↑	GF↑
DBLP			
DiTabDDPM (50 feat.)	0.229	0.999 ± 0.0	0.882 ± 0.029
DiTabDDPM (8 discr. feat.)	0.353	0.835 ± 0.090	0.794 ± 0.052
DiTabDDPM (4 discr. feat.)	0.386	0.818 ± 0.056	0.809 ± 0.051
DiTabDDPM (2 discr. feat.)	0.397	0.975 ± 0.036	0.817 ± 0.048
TabDDPM (50 feat.)	0.196	0.993 ± 0.005	0.829 ± 0.058
TabDDPM (8 discr. feat.)	0.221	0.995 ± 0.002	0.822 ± 0.052
TabDDPM (4 discr. feat.)	0.232	0.965 ± 0.046	0.825 ± 0.086
TabDDPM (2 discr. feat.)	0.235	0.979 ± 0.037	0.827 ± 0.032
VAE (baseline)	0.0	0.655 ± 0.042	0.717 ± 0.046
IMDB			
DiTabDDPM (3066 feat., all)	0.029	0.955 ± 0.084	0.890 ± 0.029
DiTabDDPM (10 discr. feat.)	0.083	0.924 ± 0.091	0.900 ± 0.042
DiTabDDPM (5 discr. feat.)	0.069	0.954 ± 0.073	0.883 ± 0.030
DiTabDDPM (2 discr. feat.)	0.040	0.976 ± 0.065	0.910 ± 0.051
TabDDPM (3066 feat., all)	0.048	0.912 ± 0.104	0.890 ± 0.036
TabDDPM (10 discr. feat.)	0.016	0.954 ± 0.073	0.903 ± 0.034
TabDDPM (5 discr. feat.)	0.011	0.988 ± 0.019	0.909 ± 0.042
TabDDPM (2 discr. feat.)	0.005	0.974 ± 0.065	0.913 ± 0.039
VAE (baseline)	0.0	0.999 ± 0.0	0.360 ± 0.165

Table 3: Comparison of explainers in terms of predictive faithfulness (PF) and ground-truth faithfulness (GF).

Approaches	DBLP		IMDB		MUTAG		BA-SHAPES		TREE-CYCLE		TREE-GRIDS		BA-3MOTIF	
	PF↑	GF↑	PF↑	GF↑	PF↑	GF↑	PF↑	GF↑	PF↑	GF↑	PF↑	GF↑	PF↑	GF↑
XGNN	0.243	0.549	0.268	0.627	0.952	0.367	0.275	0.292	0.566	0.288	0.521	0.608	0.333	0.296
GNNINTERPRETER	0.561	0.729	0.394	0.661	0.780	0.324	0.263	0.654	0.601	0.565	0.520	0.667	0.485	0.453
D4EXPLAINER	0.306	0.418	0.379	0.519	0.433	0.294	0.343	0.368	0.519	0.396	0.510	0.760	0.666	0.213
DiGNNEXPLAINER-GCN	0.641	0.800	0.410	0.850	1.0	0.425	0.315	0.766	0.525	0.716	0.507	0.893	0.432	0.639
DiGNNEXPLAINER	0.999	0.882	0.974	0.913	1.0	0.425	0.315	0.766	0.525	0.716	0.507	0.893	0.432	0.639

(as in [6]). We assess their realism by comparing the generated features to the ground truth features (Node feat. (cos)). Moreover, we evaluate the faithfulness of the resulting explanations by applying a GNN.

Implementation. We use the original model architecture for each baseline. Since the baseline explainers do not support real-valued node features or heterogeneous message passing, we convert the heterogeneous datasets DBLP and IMDB into homogeneous graphs with one-hot node types—the only setting in which all baselines can be executed. In DiGNNExplainer-GCN, we train a GCN on the

converted homogeneous versions of DBLP and IMDB, as well as on MUTAG, and synthetic datasets, whereas in DiGNNExplainer, we train GraphSAGE or HAN on the original heterogeneous versions of DBLP and IMDB, and use a GCN for the other datasets. To ensure a fair comparison, following DiGNNExplainer, each baseline utilizes a different model for each graph size. We use early stopping for all trained GNNs to prevent overfitting.

Results. In Table 3, DiGNNExplainer, which utilizes real node features, achieves superior predictive and ground-truth faithfulness compared to all baseline explainers (on DBLP, IMDB, and MUTAG). DiGNNExplainer also achieves the best ground-truth faithfulness for the synthetic datasets. On DBLP and IMDB, DiGNNExplainer-GCN performs worse than DiGNNExplainer but still outperforms all baseline explainers in both metrics.

In Table 2, for DBLP, DiTabDDPM with 50 features achieves the best explanation graphs with a maximum ground-truth faithfulness of 0.882. For IMDB, we obtain similarly high maximum scores of 0.91 for both DiTabDDPM (2 discr. feat.) and TabDDPM (2 discr. feat.). The key advantage of using ground-truth faithfulness for evaluation is robustness to spurious, out-of-distribution explanations: high GNN scores can arise from out-of-distribution graphs that would never occur in practice, whereas high ground-truth faithfulness ensures that the explanatory motifs align with the true building blocks of the data and therefore better reflect real-world behavior.

Interpretation. (1) The explanations generated by DiGNNExplainer for our real-world datasets (Table 3) are faithful, as they achieve both high predictive faithfulness and ground-truth faithfulness. (2) Our baseline explainers yield a lower ground-truth faithfulness due to their inability to model heterogeneous structures and lack of node features. (3) The low predictive faithfulness of DiGNNExplainer on the synthetic datasets arises because the 3-layer GCN is heavily biased toward predicting the base node class. As a result, only one out of the four classes (base, house, cycle, grid) is predicted correctly in many cases, which leads to a low average predictive faithfulness across classes. (4) The low ground-truth faithfulness of VAE (Table 2) suggests VAE-based graph generation is unable to preserve the underlying heterogeneous structures. (5) The high predictive faithfulness of VAE for IMDB can be explained by the fact that the GNN is trained with all features of the dataset, where all node types except movie have continuous features, and the continuous features generated using VAE are more realistic. (6) Realistic discrete features generated by learning a discrete space in DiTabDDPM result in highly faithful explanations, but we also observe that the node features for faithful explanations are not always the most realistic, i.e., do not have the

Table 4: Comparison of predictive faithfulness (PF) and ground-truth faithfulness (GF) for ablation study using a fixed graph size versus using multiple graph sizes by training a single Diffusion model.

Approach	Fixed		Multiple	
	PF $\uparrow$	GF $\uparrow$	PF $\uparrow$	GF $\uparrow$
DBLP				
DiTabDDPM (50 feat.)	0.999 $\pm$ 0.0	0.717 $\pm$ 0.040	0.999 $\pm$ 0.0	0.730 $\pm$ 0.036
DiTabDDPM (8 discr. feat.)	0.872 $\pm$ 0.102	0.787 $\pm$ 0.037	0.971 $\pm$ 0.030	0.777 $\pm$ 0.043
DiTabDDPM (4 discr. feat.)	0.770 $\pm$ 0.141	0.792 $\pm$ 0.043	0.770 $\pm$ 0.110	0.829 $\pm$ 0.071
DiTabDDPM (2 discr. feat.)	0.925 $\pm$ 0.073	0.735 $\pm$ 0.066	0.938 $\pm$ 0.071	0.812 $\pm$ 0.057
IMDB				
DiTabDDPM (3066 feat., all)	0.976 $\pm$ 0.058	0.860 $\pm$ 0.051	0.983 $\pm$ 0.021	0.853 $\pm$ 0.026
DiTabDDPM (10 discr. feat.)	0.967 $\pm$ 0.056	0.863 $\pm$ 0.069	0.972 $\pm$ 0.050	0.853 $\pm$ 0.030
DiTabDDPM (5 discr. feat.)	0.951 $\pm$ 0.087	0.886 $\pm$ 0.022	0.956 $\pm$ 0.065	0.883 $\pm$ 0.026
DiTabDDPM (2 discr. feat.)	0.902 $\pm$ 0.099	0.836 $\pm$ 0.037	0.934 $\pm$ 0.100	0.876 $\pm$ 0.033

highest cosine similarity values. (7) On DBLP (Table 2), using a larger feature size produces both high predictive faithfulness and ground-truth faithfulness scores, indicating that the GNN model can be trusted. Overall, observations (5)–(7) indicate that the node feature generation technique substantially influences the quality of explanation graphs.

4.5 Experiment on Feature Selection

In Table 2, we select varying numbers of features (2, 4, 8, 50 for DBLP and 2, 5, 10, 3066 for IMDB) using Variance Threshold. As an alternative, Table 11 in the Appendix reports results using the top k most frequent features as a selection method. Irrespective of the feature selection method, we obtain good explanation graphs, for example, with 50 features using DiTabDDPM on DBLP.

4.6 Ablation Study

We conduct an ablation study to investigate the effect of using multiple DiGress models for different graph sizes versus a fixed-size graph model. We report predictive faithfulness and ground-truth faithfulness scores for three settings: (1) a single diffusion model trained on a fixed graph size (12 for DBLP and 7 for IMDB; Table 4, left), (2) a single diffusion model trained on multiple graph sizes (10–15; Table 4, right), and (3) our main setting, which trains a separate DiGress model for each graph size (10–15; Table 2). Overall, we observe that training one model per graph size yields the best performance. In particular, for DBLP, training a single model on multiple sizes results in a lower ground-truth faithfulness of 0.730, compared to 0.882 in the main experiments.

4.7 Runtime Analysis

We compare our runtime against the baselines XGNN, GNNInterpreter, and D4Explainer (Table 5). Unlike graph generation techniques that primarily assess scalability with respect to increasing graph size, we evaluate scalability with respect to the size of the original dataset. All measurements report the total time needed for all graph sizes. Compared to D4Explainer, DiGNNExplainer is computationally faster on the large datasets DBLP and IMDB. The runtime of DiGNNExplainer remains approximately constant for

Table 5: Runtime (in secs.) comparisons for all graph sizes with baseline approaches.

Approaches	DBLP	IMDB	MUTAG	BA-SHAPES	TREE-CYCLE	TREE-GRIDS	BA-3MOTIF
XGNN	1102	363	27	351	119	155	79
GNNInterpreter	79	102	5	143	73	73	13
D4Explainer	39175	19406	658	5703	659	822	801
DiGNNExplainer	5875	4702	6910	4637	5192	5501	1992

Table 6: Breakdown of runtime (in secs.) for each component (Forest Fire Sampler (FFS), Graph Generation Training (GGT), Graph Generation After Training (GGAT), Node Feature Generation Training (NFGT), Node Feature Generation After Training (NFGAT), Postprocessing (PP), Graph selection (GS)) of DiGNNExplainer for all datasets.

	FFS	GGT	GGAT	NFGT	NFGAT	PP	GS
DBLP	7.92	5297.4	255.5	252.10	41.01	20.48	0.72
IMDB	5.08	4054.8	192.3	108.73	36.46	304.57	0.61
MUTAG	–	6699.4	210.2	–	–	–	0.83
BA-SHAPES	3.80	4380.5	252.6	–	–	–	0.69
TREE-CYCLE	3.65	4926.1	253.8	–	–	–	0.91
TREE-GRIDS	3.75	5238.4	258.5	–	–	–	0.79
BA-3MOTIF	–	1935.6	49.8	–	–	–	1.49

both small (Tree-Cycle, 871 nodes) and large (DBLP, 26128 nodes) datasets since our approach samples only small explanation graphs.

Moreover, we break down the runtime of our approach by dataset and by component (Table 6) showing that the dominant cost of DiGNNExplainer is graph generation training. For node classification datasets, we additionally run the Forest Fire Sampler (FFS) to extract training subgraphs for each explanation graph size (e.g., 10–15, i.e., six sizes). For each size, we sample $k = 200$ subgraphs, resulting in 1,200 sampled graphs in total. We empirically observe that the runtime grows from the small synthetic datasets to IMDB and DBLP, reflecting higher sampling overhead on larger underlying graphs. Next, we train DiGress on these sampled graphs. The Graph Generation Training (GGT) and Graph Generation After Training (GGAT) steps scale linearly with the number of sampled and generated graphs, respectively, as well as with the number of diffusion steps. Additionally, we train a model for node feature generation, where the runtimes for Node Feature Generation Training/After Training (NFGT/NFGAT) are only incurred for datasets with generated node attributes (here: DBLP and IMDB) and depend on the feature dimensionality and model hyperparameters. The Post-Processing (PP) step checks whether the generated graph is consistent with the metagraph. Although graph matching can be computationally hard in the worst case, this step is fast in practice for our small explanation graphs, as confirmed by our empirical runtimes. Finally, the total runtime for Graph Selection (GS), including the required GNN predictions scales linearly with the number of graphs and with the dimensionality of the node features.

4.8 Case Study

In this section, we present case study examples of explanation graphs with node features. For each class, we visualize the explanation graph that maximizes the GNN’s predicted probability for that class and corresponding feature distributions. Concretely, we show

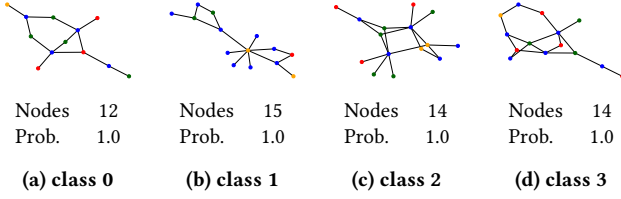


Figure 2: Explanation graphs for each class of author nodes of the DBLP dataset that maximizes the prediction of the class. Node colors indicate types: paper (blue), author (red), term (dark green), and conference (orange).

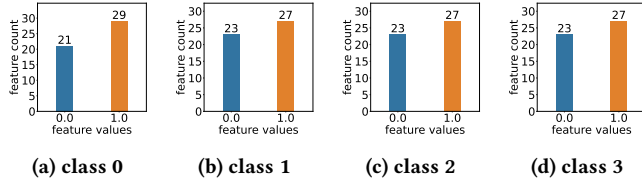


Figure 3: Frequency distribution plots visualizing the synthetic bag-of-words node feature vector of the author node of the explanation graph that is classified with maximum probability for a class.

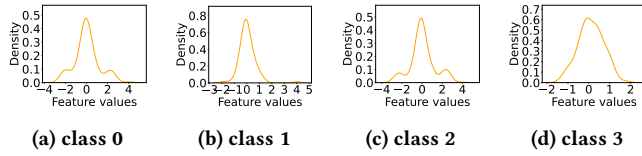


Figure 4: Density plots of the continuous term-node feature values in the explanation graphs. For each class, we show the feature-value distribution of the term nodes in the explanation graph that maximizes the predicted probability for that class.

the feature vector of the author node with the highest probability and the continuous distribution of all term nodes.

Figures 2, 3, and 4 show the explanation graphs and node features for the DiTabDDPM (50 discr. feat.) approach for DBLP. The size of the explanation graphs is between 12 and 15. All author classes are predicted with a probability of 1.0. The ground-truth faithfulness obtained for author classes is 0.8275. Figure 3 shows the distribution of the bag-of-words features (0.0/1.0) of the author node predicted with maximum probability for each class. The term nodes have continuous features (Figure 4) with a mean between 0.04 and 0.11 and a standard deviation between 0.60 and 1.22. The explanations for all approaches and all datasets can be found in our evaluation results.⁵

⁵<https://github.com/ds-jrg/DiGNNEExplainer/tree/main/evaluation>

5 Discussion

We discuss the following improvements and extensions to our approach: (1) Our approach separates the process of generating explanation graphs into two distinct steps: graph structure generation and node feature generation. This decoupling allows discrete and continuous features to be learned in their appropriate spaces, which performs better than joint continuous-space modeling. However, it does not capture advanced feature-structure dependence. We explored modeling such dependencies in DiTabDDPM and TabDDPM, but observed reduced feature realism, as quantified by cosine similarity. Therefore, we present our main results without considering advanced feature-structure dependence. In our approach, structural information is retained through node types, and feature-structure variation is captured via separate feature models per node type. (2) We obtain consistent runtime for both small and large datasets, making our approach applicable to graphs of varying sizes. Runtime could be further reduced in future work by using multiple GPUs simultaneously, as in recent DiGress implementations. (3) Training separate models for different graph sizes improves our explanations, but entails a trade-off between faithfulness and computational time. A faster alternative is to train a single diffusion model on multiple graph sizes. However, our ablation study (Table 4) shows that this yields weaker evaluation results and generates a higher number of disconnected graphs. (4) Our approach is easily extendable to real-world heterogeneous *directed* graphs, since our GNNs (GraphSAGE and HAN) can be trained on directed graphs. Moreover, DiGNNEExplainer, which builds upon DiGress, is able to support directed graphs with only minor modifications, i.e., by modifying the way noise is added to the adjacency matrix. (5) We currently obtain one explanation per class, but our implementation also supports multiple explanation graphs per class, i.e., the top k explanations ranked by softmax probabilities.

6 Conclusion

DiGNNEExplainer is the first method to explain heterogeneous GNNs on a model-level on real-world datasets using explanation graphs with actual node features. Our experiments show that incorporating actual node features significantly improves explanation faithfulness: DiGNNEExplainer achieves higher predictive and ground-truth faithfulness than state-of-the-art baselines, while producing explanation graphs whose statistical properties closely match those of real-world graphs. This is enabled by (1) node feature generation via our novel discrete diffusion model, DiTabDDPM, which enables the synthesis of realistic discrete features; and (2) our discrete diffusion-based approach for modeling heterogeneous graphs, supporting the generation of structurally realistic explanation graphs. Beyond the citation and movie datasets used in our experiments, DiGNNEExplainer is broadly applicable to any domain with heterogeneous graph data and structured node features, such as healthcare, finance, or chemistry. Further runtime optimization is left for future work.

Acknowledgments

This work has been supported by the European Union and the Ministry of Economic Affairs, Industry, Climate Action and Energy of the State of North Rhine-Westphalia (MWIKE NRW) under the grant no EFRE-20801043.

References

- [1] Chirag Agarwal, Satyapriya Krishna, Eshika Saxena, Martin Pawelczyk, Nari Johnson, Isha Puri, Marinka Zitnik, and Himabindu Lakkaraju. 2022. OpenXAI: Towards a Transparent Evaluation of Model Explanations. In *NeurIPS*.
- [2] Chirag Agarwal, Owen Queen, Himabindu Lakkaraju, and Marinka Zitnik. 2023. Evaluating Explainability for Graph Neural Networks. *Scientific Data* 10, 1 (2023), 144.
- [3] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.
- [4] Jialin Chen, Shirley Wu, Abhijit Gupta, and Rex Ying. 2023. D4Explainer: In-Distribution GNN Explanations via Discrete Denoising Diffusion. *CoRR* abs/2310.19321 (2023).
- [5] Caglar Demir, Alkid Baci, N'Dah Jean Kouagou, Leonie Nora Sieger, Stefan Heindorf, Simon Bin, Lukas Blübaum, Alexander Biggerl, and Axel-Cyrille Ngonga Ngomo. 2025. Ontolearn—A Framework for Large-scale OWL Class Expression Learning in Python. *Journal of Machine Learning Research* 26, 63 (2025), 1–6.
- [6] Wei Dong, Junsheng Wu, Zongwen Bai, Yaoqi Hu, Weigang Li, Wei Qiao, and Marcin Wozniak. 2021. MobileGCN applied to low-dimensional node feature learning. *Pattern Recognit.* 112 (2021), 107788.
- [7] Chi Thang Duong, Thanh Dat Hoang, Ha The Hien Dang, Quoc Viet Hung Nguyen, and Karl Aberer. 2019. On Node Features for Graph Neural Networks. *CoRR* abs/1911.08795 (2019).
- [8] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding. In *WWW. ACM / IW3C2*, 2331–2341.
- [9] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting Deep Learning Models for Tabular Data. In *NeurIPS*. 18932–18943.
- [10] William L. Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NIPS*. 1024–1034.
- [11] Stefan Heindorf, Lukas Blübaum, Nick Düsterhus, Till Werner, Varun Nandkumar Golani, Caglar Demir, and Axel-Cyrille Ngonga Ngomo. 2022. EvoLearner: Learning Description Logics with Evolutionary Algorithms. In *WWW. ACM*, 818–828.
- [12] Stefan Heindorf and Daniel Neib. 2025. Assessing Natural Language Explanations of Relational Graph Neural Networks. In *CIKM. ACM*, 4785–4789.
- [13] Anna Himmelhuber, Stephan Grimm, Sonja Zillner, Mitchell Joblin, Martin Ringsquandl, and Thomas A. Runkler. 2021. Combining Sub-symbolic and Symbolic Methods for Explainability. In *RuleML+RR (Lecture Notes in Computer Science, Vol. 12851)*. Springer, 172–187.
- [14] Jaehyeon Jo, Seul Lee, and Sung Ju Hwang. 2022. Score-based Generative Modeling of Graphs via the System of Stochastic Differential Equations. In *ICML (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 10362–10383.
- [15] Dominik Köhler and Stefan Heindorf. 2024. Utilizing Description Logics for Global Explanations of Heterogeneous Graph Neural Networks. *CoRR* abs/2405.12654 (2024).
- [16] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. 2023. TabDDPM: Modelling Tabular Data with Diffusion Models. In *ICML (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 17564–17579.
- [17] Chen Ling, Carl Yang, and Liang Zhao. 2021. Deep Generation of Heterogeneous Networks. In *ICDM. IEEE*, 379–388.
- [18] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized Explainer for Graph Neural Network. In *NeurIPS*.
- [19] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. *CoRR* abs/2007.08663 (2020).
- [20] Alexander Pluska, Pascal Welke, Thomas Gärtner, and Sagar Malhotra. 2024. Logical Distillation of Graph Neural Networks. In *KR*.
- [21] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. 2020. Little Ball of Fur: A Python Library for Graph Sampling. In *CIKM. ACM*, 3133–3140.
- [22] Rupesh Sapkota, Dominik Köhler, and Stefan Heindorf. 2024. EDGE: Evaluation Framework for Logical vs. Subgraph Explanations for Node Classifiers on Knowledge Graphs. In *CIKM. ACM*, 4026–4030.
- [23] Martin Simonovsky and Nikos Komodakis. 2018. GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders. In *ICANN (1) (Lecture Notes in Computer Science, Vol. 11139)*. Springer, 412–422.
- [24] Arjun Subramanian. 2021. MOTIF-Driven Contrastive Learning of Graph Representations. In *AAAI AAAI Press*, 15980–15981.
- [25] Clément Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. 2023. DiGress: Discrete Denoising diffusion for graph generation. In *ICLR. OpenReview.net*.
- [26] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. 2019. Heterogeneous Graph Attention Network. In *WWW. ACM*, 2022–2032.
- [27] Xiaoqi Wang and Han-Wei Shen. 2023. GNNInterpreter: A Probabilistic Generative Model-Level Explanation for Graph Neural Networks. In *ICLR*.
- [28] Xiang Wang, Ying-Xin Wu, An Zhang, Xiangnan He, and Tat-Seng Chua. 2021. Towards Multi-Grained Explainability for Graph Neural Networks. In *NeurIPS*. 18446–18458.
- [29] Zhitaoying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GNNExplainer: Generating Explanations for Graph Neural Networks. In *NeurIPS*. 9240–9251.
- [30] Zhaoning Yu and Hongyang Gao. 2022. MotifExplainer: a Motif-based Graph Neural Network Explainer. *CoRR* abs/2202.00519 (2022). arXiv:2202.00519 https://arxiv.org/abs/2202.00519
- [31] Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. 2020. XGNN: Towards Model-Level Explanations of Graph Neural Networks. In *KDD. ACM*, 430–438.
- [32] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On Explainability of Graph Neural Networks via Subgraph Explorations. In *ICML (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 12241–12252.
- [33] Yuchen Zhang, Xiaoxiao Ma, Jia Wu, Jian Yang, and Hao Fan. 2024. Heterogeneous Subgraph Transformer for Fake News Detection. In *WWW. ACM*, 1272–1282.
- [34] Zheng Zhang, Yingsheng Ji, Jiachen Shen, Yushu Chen, Xi Zhang, and Guangwen Yang. 2024. Collaborative Metapath Enhanced Corporate Default Risk Assessment on Heterogeneous Graph. In *WWW. ACM*, 446–456.

A Diffusion Models

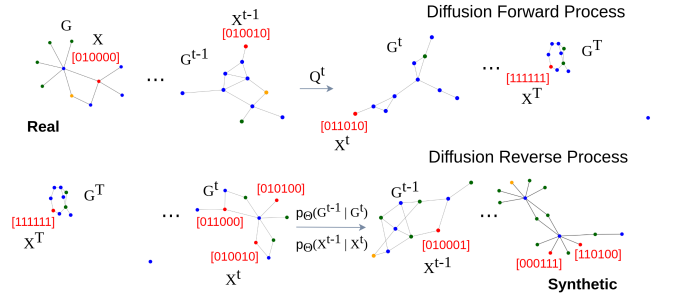


Figure 5: Diffusion steps for generation of synthetic graph and discrete node features.

A.1 DiGress

DiGress [25] is a discrete diffusion approach for graph generation, which adds noise through discrete time steps based on a Markov process. The noise model is a Markov process over a discrete state-space. In a discrete state-space, each node and edge transitions to a different state with the addition of noise at every time step. A state refers to a node type or an edge type. Following each transition, the node or edge changes to another type. The probabilities of transition from one state to another are represented as transition matrices for nodes and for edges. In DiGress, the next step G_t (Figure 5) is obtained by the product of the transition matrix, Q_t at step t and the graph at the previous step, G_{t-1} . The transition probability, q at G_t is given as:

$$q(G_t|G_{t-1}) = G_{t-1}Q_t \quad (1)$$

at time step $t - 1$. The noise at step t is calculated as the product of transition matrices for t steps. DiGress adds noise using a cosine noise scheduler for t steps. At the end of t steps, transition matrices for node types and edge types are obtained. The transition probabilities at step t represent the complete noise distribution for each node and edge. Diffusion is applied to nodes and edges independently by adding marginal noise to each node and edge. The noise model adds noise to an input until the input changes

to approximately pure noise. An input node feature vector, X , follows the same noise model to transition to the previous step, X_{t-1} . The pure noise is represented as a uniform distribution, which is the initial prior distribution for the denoising network, p_θ , during the reverse process. The denoising network in DiGress is made of GraphTransformer layers. Each layer has a multi-headed attention module. The denoising network uses self-attention to recover the input distribution by predicting the noise added at each previous step. The training loss, L_{Graph} for graph is given as

$$L_{Graph} = L_{Nodes} + \lambda L_{Edges}$$

where node loss, L_{Nodes} and edge loss, L_{Edges} represent the cross-entropy loss between the true node and edge, and predicted probabilities of each node and edge, and $\lambda \in \mathbb{R}^+$ assigns relative importance to nodes and edges. The graph loss is computed as a combined loss of individual node loss and edge loss. The loss of each node and edge is measured using cross-entropy. DiGress models node features along with the graph structure. The node features are one-hot encoding of node types. Extra features are added to nodes, including structural information that encodes the presence of cycles, and spectral features for positional encodings needed for a transformer model. These extra features are added at each diffusion step, as the message passing networks are unable to capture the presence of cycles. DiGress performs well at generating small heterogeneous graphs with the same node size as the input graph. Discrete node types and edge types are sampled from the complete noise distributions. The samples are input to a denoising network, which uses self-attention to predict the noise and outputs the distributions for node types and edge types for each previous step.

A.2 TabDDPM

TabDDPM [16] extends the denoising diffusion probabilistic (DDPM) approach used for images to synthesize tabular data. In TabDDPM, an input feature vector, $x = [x_{num}, x_{cat_1} \dots x_{cat_C}]$ consists of both numeric features, x_{num} and categorical features, x_{cat_i} with $i \in C$ categories. The categorical features are represented as one-hot encoding of categories. TabDDPM models categorical and binary features using multinomial diffusion and numerical features using continuous noise. The forward process adds Gaussian noise using a Markov process, while the reverse process uses a denoising neural network consisting of MLP layers with SiLU non-linear activation. The denoising network is an MLP architecture of the form [9]:

$$MLP(x) = \text{Linear}(MLPBlock(\dots(MLPBlock(x)))) \quad (2)$$

where $MLPBlock(x) = \text{Dropout}(\text{ReLU}(\text{Linear}(x)))$. The TabDDPM method outperforms GAN and VAE in synthetic tabular data generation. TabDDPM captures training loss using mean-squared error and KL-Divergence. Our modification of this approach for generating node features uses discrete noise to model discrete features and cross-entropy to measure node feature loss during training.

B Additional Details for DiGNNExplainer

B.1 DiTabDDPM - Denoising network

Our denoising network

$$\begin{aligned} \text{MLPOut}(x) &:= \text{Lin}(\text{SiLU}(\dots \\ &\quad \text{Lin}(\text{SiLU}(\text{Lin}(\text{Layers})))) \end{aligned}$$

consists of MLPBlock layers

$$\text{Layers} := \text{MLPBlock}(\dots(\text{MLPBlock}(\text{MLPIn})))$$

$$\begin{aligned} \text{MLPIn} &:= \text{Lin}(\text{ReLU}(\text{SiLU}(\dots \\ &\quad \text{Lin}(\text{ReLU}(\text{SiLU}(x)))))) \end{aligned}$$

$$\begin{aligned} \text{MLPBlock} &:= \text{Lin}(\text{Drop}(\text{SiLU}(\text{Lin} \\ &\quad (\text{Drop}(\text{MLPIn})))) \end{aligned}$$

where x denotes the discrete feature vector for a single node, Lin a linear layer, SiLU a sigmoid linear unit, ReLU a rectified linear unit, and Drop a dropout layer. An MLPBlock consists of normalization and dropout layers with a combination of ReLU and SiLU activations. Normalization is added to the input and output of each MLPBlock layer. The output of an MLPBlock layer is the input for the next MLPBlock layer. In a pilot study, we experimented with different architectures, including different normalization layers and activation functions. We found the given architecture to yield the smallest loss on the validation set.

B.2 Node Feature Reconstruction Loss

The node feature reconstruction loss, L_{recon} , at validation time, is computed using the sampled features, x_0 , drawn from the probability distributions at step 0, and x_t , drawn from the predicted probability distribution of the denoising network θ , at step t . It is given as the expected value of the logarithm of the predicted probability of the input features given the noisy features at step t .

$$L_{recon} = \mathbb{E} [\log p_\theta(x_0|x_t)]$$

B.3 Postprocessing

Two nodes τ_1, τ_2 in the metagraph are connected if and only if there exist two nodes v_1 and v_2 in the graph such that v_1 has type τ_1 and v_2 has type τ_2 . The generated graph is consistent if and only if the metagraph of the generated graph is a subgraph of the metagraph of the dataset.

B.4 Explanation Graph Selection

Let $\mathcal{G}$ be the set of valid graphs obtained in the above step (Postprocessing). Given a *graph classification task*, for every graph $G \in \mathcal{G}$, we make a prediction $\hat{y}_G := f(G)$ where $\hat{y}_G \in [0, 1]^C$ denotes the vector of predicted softmax probabilities and f our trained GNN. Then, for every class c_i with $i \in \{1, \dots, C\}$, we take the graph

$$G_i := \arg \max_{G \in \mathcal{G}} \hat{y}_{G,i}$$

with the highest predicted score as the explanation graph for class c_i . Similarly, given a *node classification task*, where we need to make a prediction for every node of type τ , for every valid graph $G \in \mathcal{G}$ and every node $v \in G$ that has type τ , we make a prediction $\hat{y}_{(G,v)} := f(G, v)$ where $\hat{y}_{(G,v)} \in [0, 1]^C$ denotes the vector of predicted softmax probabilities and f our GNN. Then, for every class c_i

Table 7: Dataset statistics: number of graphs, average number of nodes per graph, number of classified nodes (e.g., author nodes for DBLP), number of classified graphs, number of features of the node to be classified, number of classes to be predicted, number of continuous features, and number of discrete features.

Datasets	Graphs	Nodes (avg.)	Nodes (classified)	Graphs (classified)	Feat.	Classes	Cont. (feat.)	Discr. (feat.)
DBLP	1	26,128	4,057	–	334	4	50	4,565
IMDB	1	11,616	4,278	–	3,066	3	6,132	3,066
MUTAG	188	17	–	188	–	2	–	–
BA-SHAPES	1	700	700	–	–	4	–	–
TREE-CYCLE	1	871	871	–	–	2	–	–
TREE-GRIDS	1	1,231	1,231	–	–	2	–	–
BA-3MOTIF	3,000	21	–	3,000	–	3	–	–

with $i \in \{1, \dots, C\}$, we take the graph

$$G_i := \arg \max_{G \in \mathcal{G}} \max_{v \in G, v \text{ has type } \tau} \hat{y}_{(G,v),i}$$

with the highest predicted score as the explanation graph for class c_i . This way, we obtain one explanation graph per class.

C Datasets

We run experiments for DiGNNExplainer on the following datasets in Table 7: DBLP [8] is a heterogeneous dataset of a citation network with four node types—author, paper, term, and conference. Author node features are bag-of-words representations of their paper keywords. Paper nodes have bag-of-words features, term nodes have continuous features, and conference nodes have no features. IMDB [8] is a heterogeneous dataset for movies with three node types—movie, director, and actor. The movie node features include bag-of-words representations of their plot keywords. Director and actor nodes have continuous features. MUTAG [19] is a molecule graph classification dataset where each graph can be classified into two classes—mutagenic and non-mutagenic, and each node has a single node type, atom. The synthetic BA-SHAPES [29] dataset is constructed by randomly attaching house motifs to the nodes of a Barabási–Albert (BA) base graph. All nodes are of a single type. TREE-CYCLE [29] and TREE-GRIDS [29] are constructed by randomly attaching cycle motifs and n-by-n grid motifs, respectively, to the nodes of a balanced binary tree base graph. BA-3MOTIF [28] is a motif graph classification dataset where a house/cycle/grid motif is attached randomly to the nodes of Barabási–Albert (BA) base graphs. The graphs can be classified into any of three types, as per the type of motif attached.

D Validity Analysis

We observe that a higher number of graphs generated using DiGNNExplainer are valid (i.e., consistent with the metagraph of the dataset) for DBLP compared to IMDB (Table 8). For homogeneous datasets, the number of connected graphs is the same as valid graphs, as homogeneous graphs do not require validation using a metagraph.

Table 8: Analysis of the validity of generated graphs according to the metagraph.

Approach	#Graphs (Generated)	#Graphs (Connected)	#Graphs (After validity check)	Avg. nodes per graph
DBLP				
DiGNNExplainer	1536	1416	231	12.5
VAE(baseline)	1536	1283	210	12.5
IMDB				
DiGNNExplainer	1536	1367	188	7.92
VAE(baseline)	1536	948	513	7.92
MUTAG				
DiGNNExplainer	1536	731	731	13
VAE(baseline)	1536	1242	1242	13
BA-SHAPES				
DiGNNExplainer	1536	1058	1058	12.5
VAE(baseline)	1536	1323	1323	12.5
TREE-CYCLE				
DiGNNExplainer	1536	1398	1398	12.5
VAE(baseline)	1536	1237	1237	12.5
TREE-GRIDS				
DiGNNExplainer	1536	1196	1196	12.5
VAE(baseline)	1536	1312	1312	12.5
BA-3MOTIF				
DiGNNExplainer	1536	1525	1525	15
VAE(baseline)	1536	1369	1369	15

E Community Detection

We use communities or motifs to evaluate the ground-truth faithfulness of the explanation graphs (see Section 4.1), following previous work [29, 31]. Table 9 (right) shows example motifs.

For *real-world* graphs (DBLP, IMDB, MUTAG), we employ Louvain community detection⁶ [3] which identifies the largest substructures in input graphs. Unlike existing techniques [24, 30] that often rely on blackbox models for motif detection, our method is more similar to GraphXAI [2], which detects ground-truth substructures in the real-world dataset MUTAG via substructure matching. For *synthetic* graphs (BA-3MOTIF, BA-SHAPES, TREE-CYCLE, and TREE-GRIDS), we generate motifs using ShapeGGen [2].

We use *class-specific* motifs when motifs can be associated with a prediction class, and *common* motifs otherwise. For the *graph classification datasets* (MUTAG, BA-3MOTIF), each graph belongs to a single class, so we construct class-specific motif sets. Similarly, for the *real-world node classification datasets* DBLP and IMDB, we obtain class-specific motifs. In DBLP, most (104 out of 120) of the sampled graphs using ForestFireSampler contain author nodes from a single class, in IMDB, 40 out of 143 graphs contain movie nodes from a single class. If a sampled graph has nodes of different classes (e.g., containing author nodes from classes 1 and 3), we do not use it for motif detection, because the obtained motifs might be common across classes. In this way, we detect motifs that help to distinguish different classes.

In contrast, for the *synthetic node classification datasets* BA-SHAPES, TREE-CYCLE, and TREE-GRIDS, the task is to predict node labels and every graph contains nodes of all labels. For example, in BA-SHAPES, a house motif has all four node labels. As a result,

⁶<https://python-louvain.readthedocs.io/en/latest/api.html>

Table 9: Explanation graphs for Class 1

	XGNN	GNNInterpreter	D4Explainer	DiGNNExplainer	Motifs
DBLP					
IMDB					
MUTAG					
BA-SHAPES					
TREE-CYCLE					
TREE-GRIDS					
BA-3MOTIF					

Table 10: Motif statistics. Top: For datasets with *class-specific* motifs, we report, for each class, the number of detected motifs that *occur* and *do not occur* in the explanation graphs. Bottom: For datasets with common motifs, we report the total number of motifs and how many motifs appear in the best explanation graph of *every* class, in at least $\frac{1}{2}$ of the classes, and in *none* of the classes.

Dataset	Class 0		Class 1		Class 2		Class 3	
	occur	not occur	occur	not occur	occur	not occur	occur	not occur
DBLP	8	2	8	2	10	0	8	2
IMDB	9	1	9	1	10	0	–	–
MUTAG	30	14	3	11	–	–	–	–
BA-3MOTIF	7	3	2	8	10	0	–	–

Dataset	No. of motifs	No. of motifs (every class)	No. of motifs ($\frac{1}{2}$ of the classes)	No. of motifs (none of the classes)
BA-SHAPES	30	14	26	4
TREE-CYCLE	30	15	28	2
TREE-GRIDS	30	28	30	0

motifs are naturally shared across labels/classes, and we evaluate faithfulness using a single set of *common* motifs. Table 10 shows the statistics for class-specific and common motifs for our datasets.

F Case Study—Explanations for the same class

Table 9 shows a qualitative analysis of the explanation graphs obtained for the same class (class 1) for all datasets and all approaches, along with example motifs for each dataset.

G Feature Selection by Frequency

Table 11 reports predictive faithfulness (PF), ground-truth faithfulness (GF), and cosine similarity (Node Feat. (CS)) for frequency-based feature selection.

Table 11: Feature selection by frequency

Approach	Node feat.(cos) $\uparrow$	PF $\uparrow$	GF $\uparrow$
DBLP			
DiTabDDPM (50 feat.)	0.233	0.902 $\pm$ 0.102	0.867 $\pm$ 0.046
DiTabDDPM (8 discr. feat.)	0.355	0.797 $\pm$ 0.062	0.852 $\pm$ 0.054
DiTabDDPM (4 discr. feat.)	0.387	0.770 $\pm$ 0.121	0.820 $\pm$ 0.055
DiTabDDPM (2 discr. feat.)	0.384	0.972 $\pm$ 0.041	0.845 $\pm$ 0.052
TabDDPM (50 feat.)	0.194	0.998 $\pm$ 0.001	0.842 $\pm$ 0.054
TabDDPM (8 discr. feat.)	0.221	0.976 $\pm$ 0.034	0.852 $\pm$ 0.052
TabDDPM (4 discr. feat.)	0.310	0.916 $\pm$ 0.072	0.857 $\pm$ 0.057
TabDDPM (2 discr. feat.)	0.235	0.969 $\pm$ 0.038	0.822 $\pm$ 0.042
VAE (baseline)	0.0	0.664 $\pm$ 0.048	0.717 $\pm$ 0.044
IMDB			
DiTabDDPM (3066 feat., all)	0.029	0.955 $\pm$ 0.084	0.890 $\pm$ 0.029
DiTabDDPM (10 discr. feat.)	0.083	0.977 $\pm$ 0.066	0.933 $\pm$ 0.036
DiTabDDPM (5 discr. feat.)	0.069	0.970 $\pm$ 0.065	0.900 $\pm$ 0.036
DiTabDDPM (2 discr. feat.)	0.040	0.984 $\pm$ 0.039	0.913 $\pm$ 0.047
TabDDPM (3066 feat., all)	0.048	0.912 $\pm$ 0.104	0.890 $\pm$ 0.036
TabDDPM (10 discr. feat.)	0.016	0.998 $\pm$ 0.003	0.893 $\pm$ 0.044
TabDDPM (5 discr. feat.)	0.011	0.973 $\pm$ 0.068	0.896 $\pm$ 0.040
TabDDPM (2 discr. feat.)	0.005	0.968 $\pm$ 0.065	0.909 $\pm$ 0.039
VAE (baseline)	0.0	0.999 $\pm$ 0.005	0.360 $\pm$ 0.165

H Hyperparameters

For GNN training, we use the following settings. GraphSAGE is trained with a learning rate of 0.005 and a weight decay of 0.001. HAN is trained with a dropout of 0.6, a learning rate of 0.005, and a weight decay of 0.001. For GCN training, on MUTAG, we use a dropout of 0.3 and a learning rate of 0.0005, whereas for synthetic datasets we use 0.5 and 0.005, respectively.

For graph generation with diffusion models, we use a batch size of 256, a learning rate of 0.0002, a weight decay of $1e-12$, and 5,000 diffusion steps. For DiTabDDPM, we use a batch size of 32, a learning rate of 0.05, a weight decay of 0.001, and 25,000 diffusion steps. For TabDDPM, we use the same hyperparameters as in the original implementation.

I Implementation

All code is implemented in Python 3.10.6. We utilize Pandas for data manipulation, Matplotlib and Seaborn for visualization, NetworkX for graph plotting, PyTorch Geometric for GNN training, scikit-learn for sampling node features and comparing feature distributions, the Forest Fire Sampler for sampling subgraphs, the Deep Graph Library (DGL) for creating metagraphs, and Python-louvain for community detection.

The experiments were conducted on a machine with Ubuntu 22.04.2 LTS, an AMD Ryzen 9 CPU, and an NVIDIA GeForce RTX 3070 GPU with 8GB of GPU memory.